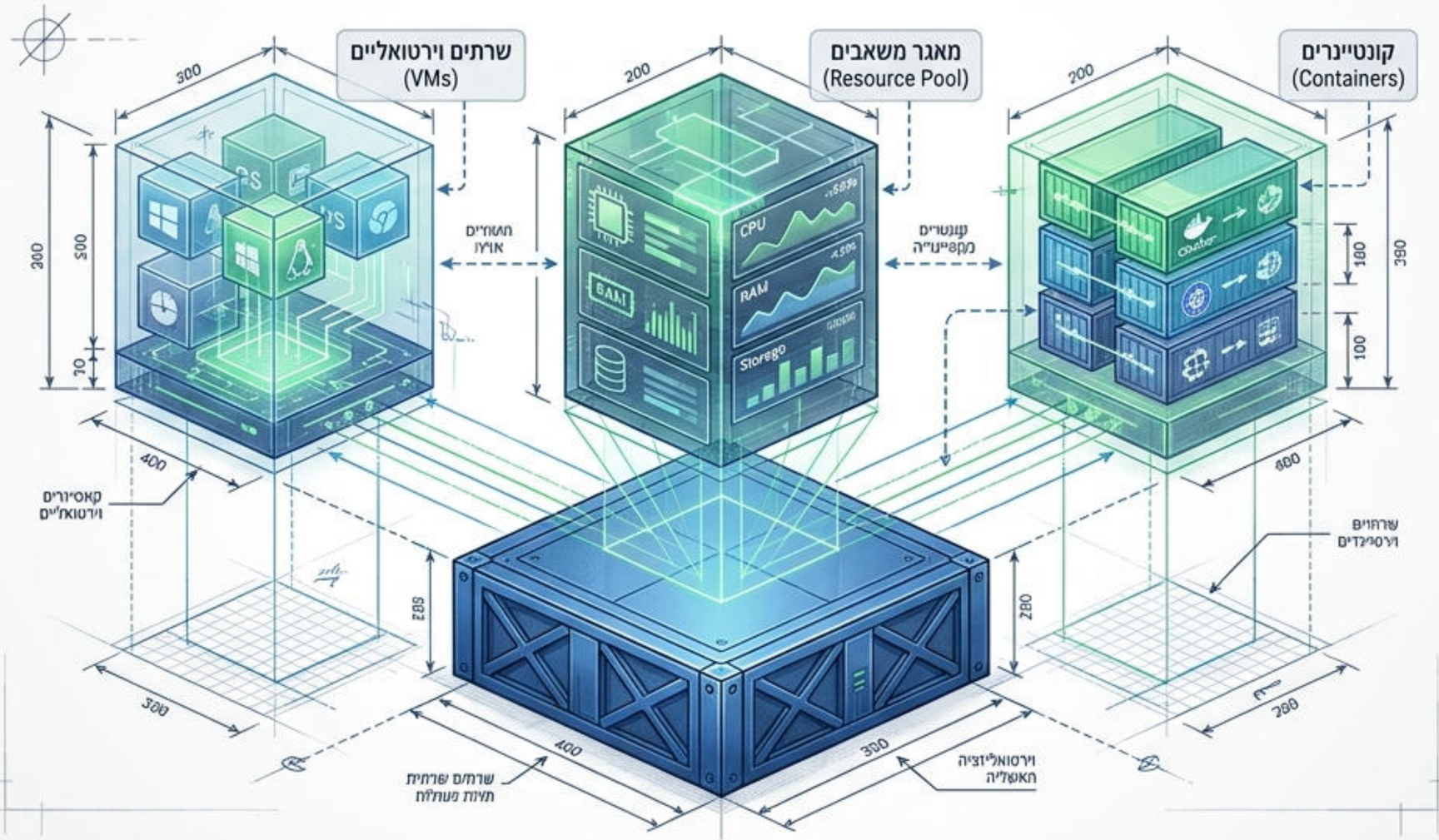
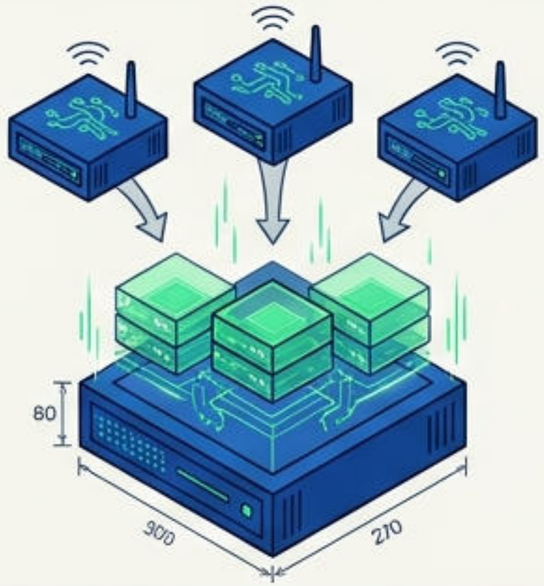
וירטואליזציה וענן: המדריך המקיף

מאיחוד שרתים ועד קונטיינרים – ארכיטקטורה, חומרה ואשליה



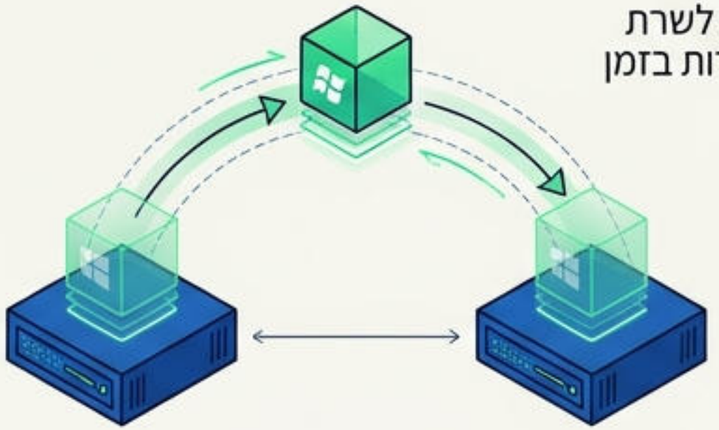
1. איחוד שרתים (Server Consolidation)

חיסכון דרמטי בעלויות חומרה וחשמל על ידי אריזת שרתים ייעודיים כמכונות וירטואליות על שרת פיזי יחיד.



2. ניוד מערכות (Live Migration)

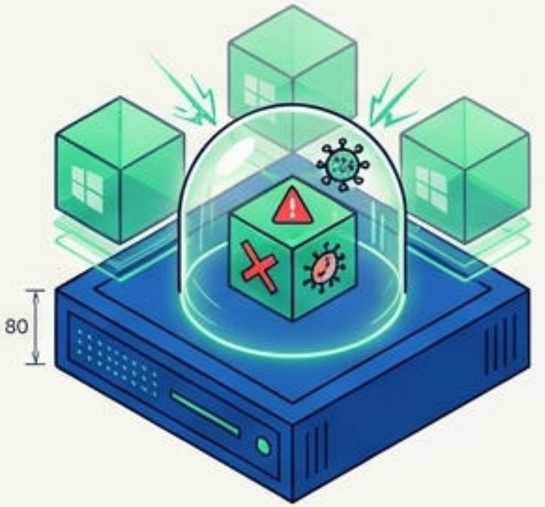
ייצוג המכונה כאוסף קבצים מאפשר העברתה משרת לשרת ללא כיבוי או הפסקת שירות בזמן תחזוקה.





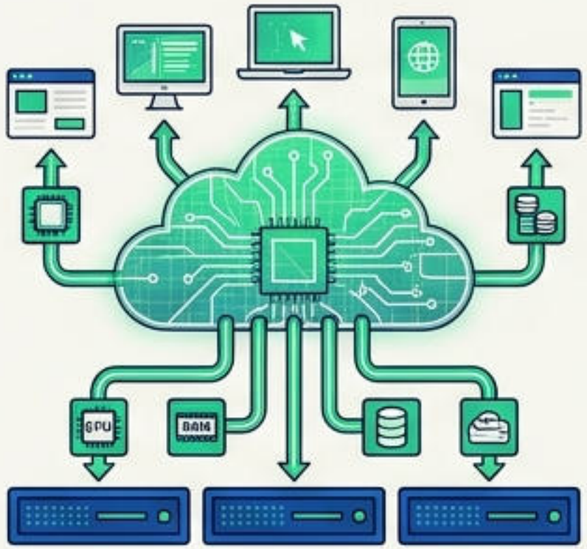
3. מידור ובטיחות (Isolation)

קריסת מערכת או וירוס במכונה אחת אינם משפיעים כלל על המארח או על מכונות שכנות באותו שרת.



4. מחשוב ענן (Cloud Computing)

השכרת משאבים מבוססי-דרישה ללקוחות מרובים תוך שיתוף יעיל של התשתית הפיזית.



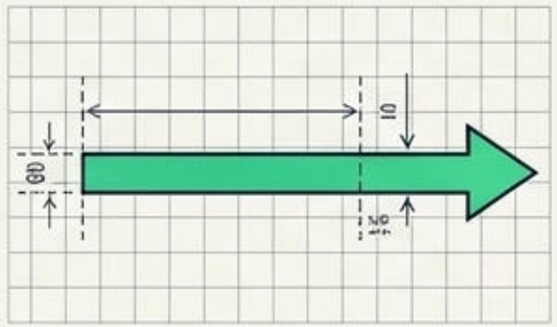
https://operating-system-5l3.pages.dev/

רמת מערכת ההפעלה (Containers)



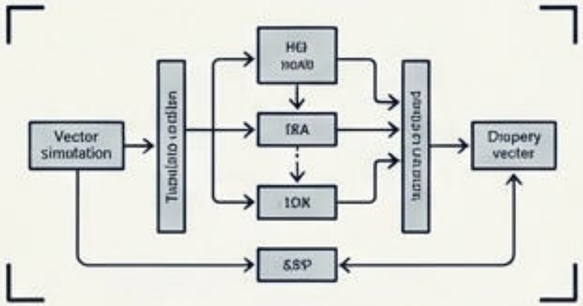
אין חומרה וירטואלית.
הליבה מפצלת משאבים לסביבות מבודדות.
מאפיין: קל משקל ומהיר.

רמת ה-HAL (Hypervisor)

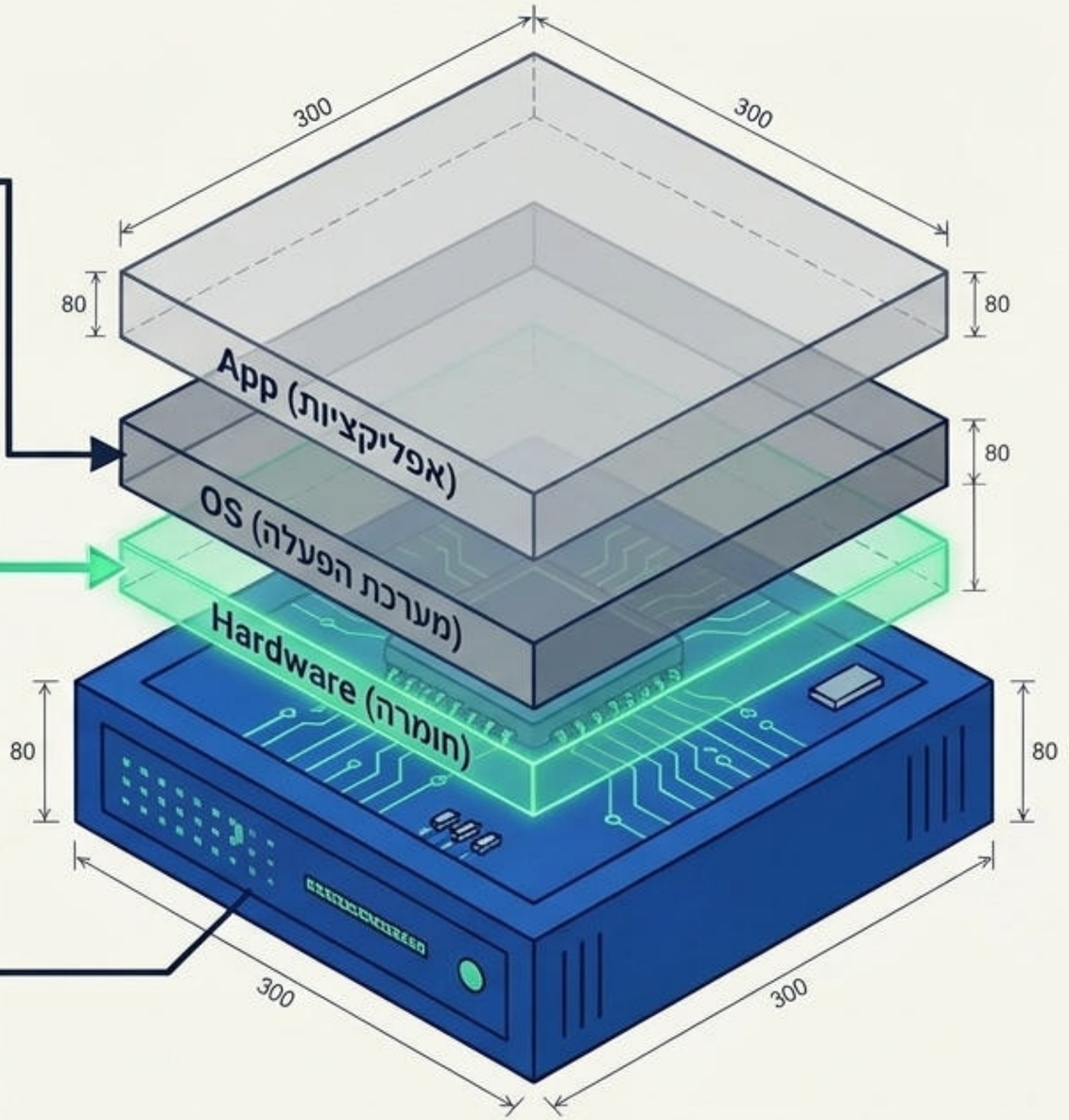


שכבת הפשטת החומרה. הרצת פקודות ישירות על המעבד הפיזי לביצועים אופטימליים.
(מוקד המצגת)

רמת ה-ISA (Emulation)



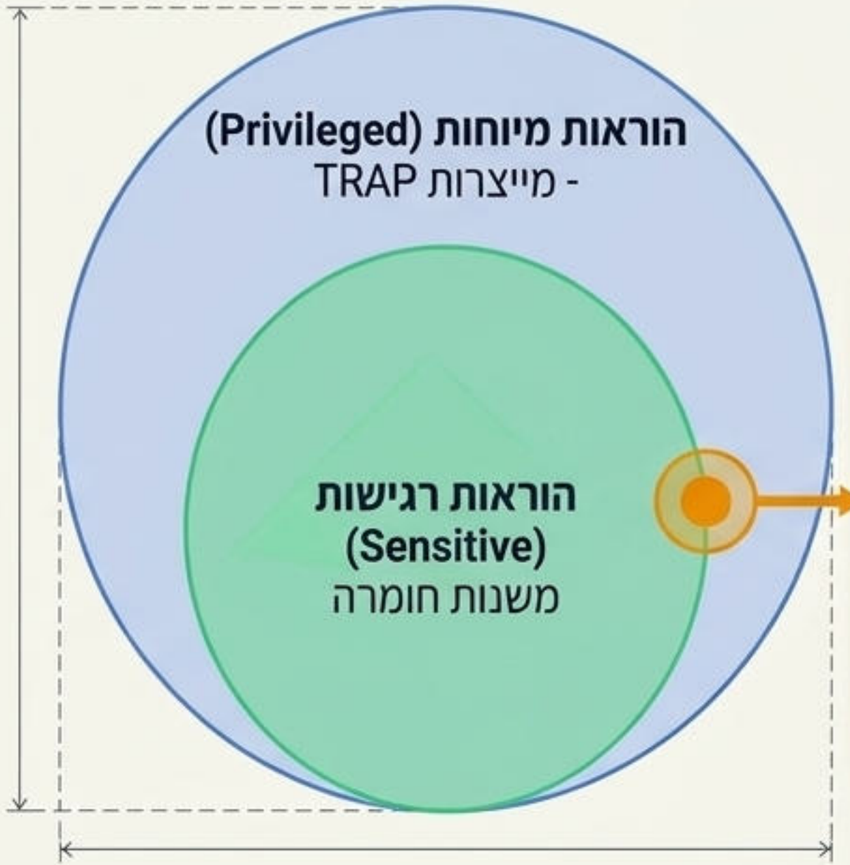
חיקוי חומרה מלא. תוכנה המתרגמת כל פקודת מכונה.
מאפיין: איטי מאוד, מיועד לתאימות.



חוקי המשחק והכשל של ארכיטקטורת x86

שלוש דרישות היסוד (Popek-Goldberg)

1. **נאמנות למקור (Fidelity)** - תוצאות זהות לחומרה אמיתית.
2. **שליטה במשאבים (Resource Control)** (Resource Control) - ה-Hypervisor הוא הבוס הבלעדי.
3. **יעילות (Efficiency)** - הרצה ישירה ללא חיקוי תוכנתי - הרצוה ישירה ללא (Trap-and-Emulate).



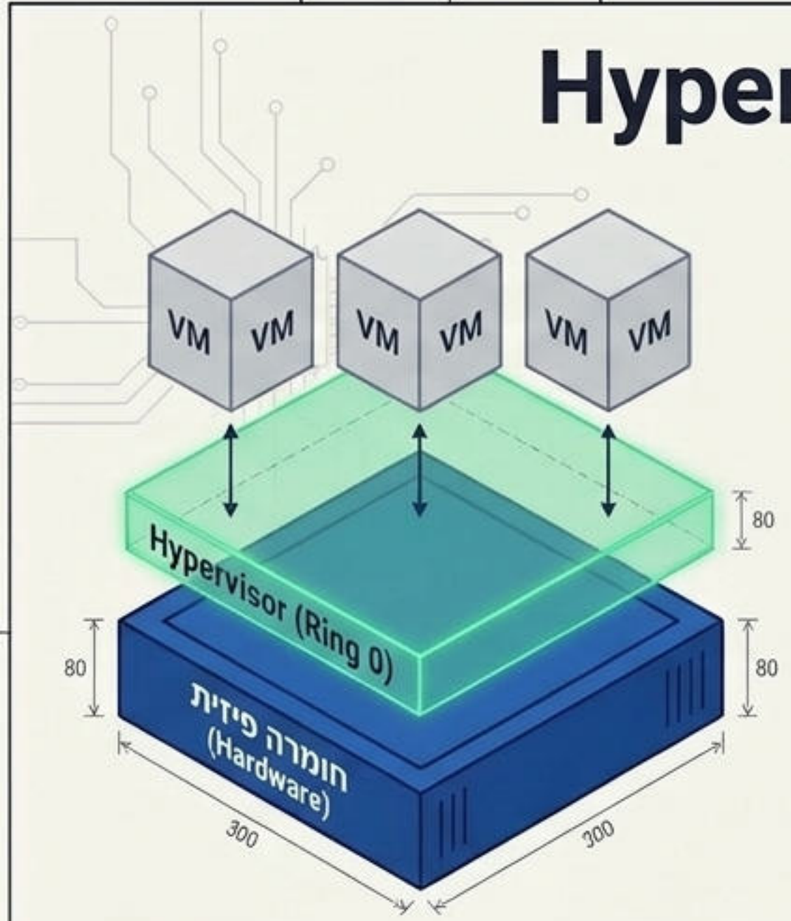
החור של ארכיטקטורת x86

פקודות כמו POPF הן רגישות אך לא מיוחסות. הניסיון לבצען נכשל שקט (Silent Failure) ללא ייצור TRAP, מה ששובר לחלוטין את האשליה הווירטואלית.

Sensitive ⊆ Privileged

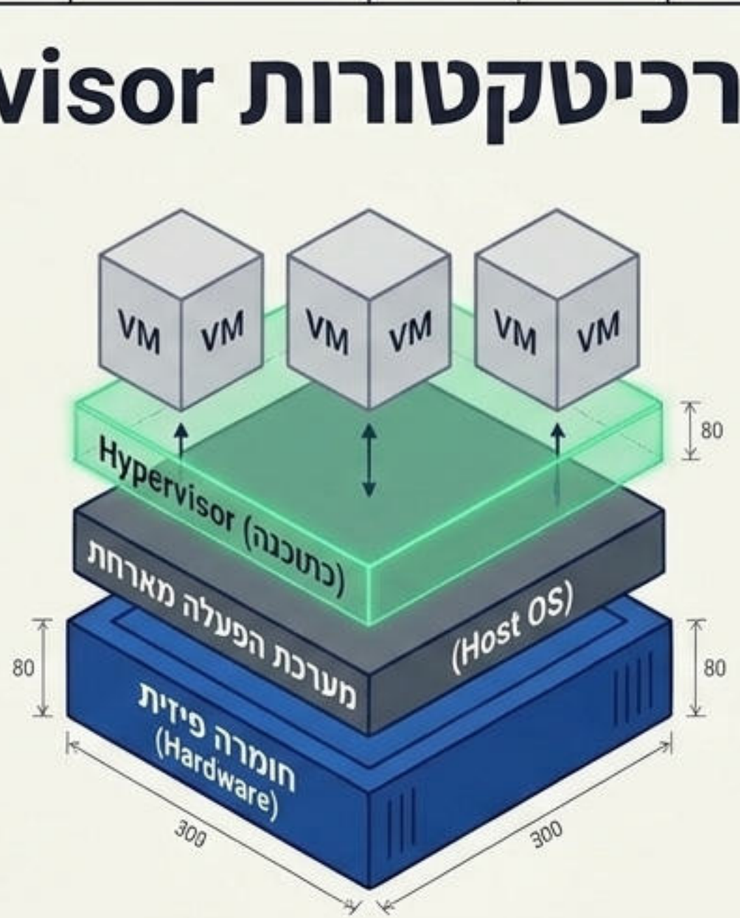
https://operating-system-5l3.pages.dev/

סיווג ארכיטקטורות Hypervisor



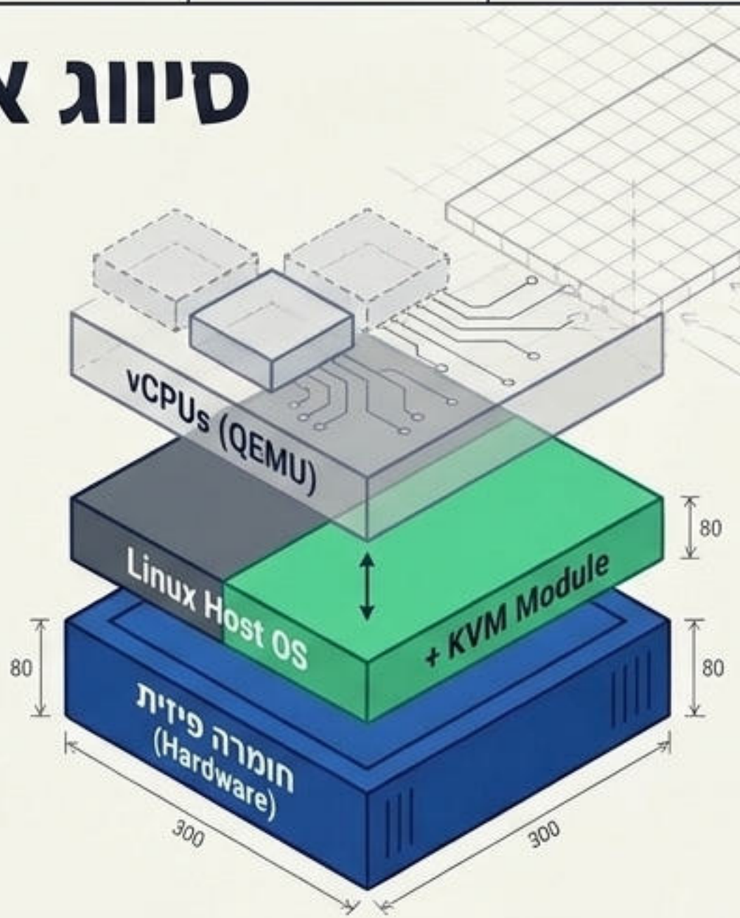
טיפוס 1 (Bare-Metal)

מערכת ההפעלה היא ה-Hypervisor עצמו. מותקן ישירות על הברזל ליעילות מקסימלית באנטרפרייז. שולט בדרייברים עצמאית (דוגמה: VMware ESXi).



טיפוס 2 (Hosted)

מותקן כתוכנה על גבי מערכת הפעלה רגילה ומסתייע בדרייבר קרנל. סובל מתקורה עקב ריבוי שכבות אך נוח למפתחים (דוגמה: VirtualBox).



גישת ההכלאה (KVM)

הופך את הקרנל של לינוקס לטיפוס 1 ישירות על החומרה, אך שומר על תזמון תהליכים גמיש של טיפוס 2. התקן המוביל בענן המודרני.

https://operating-system-5l3.pages.dev/

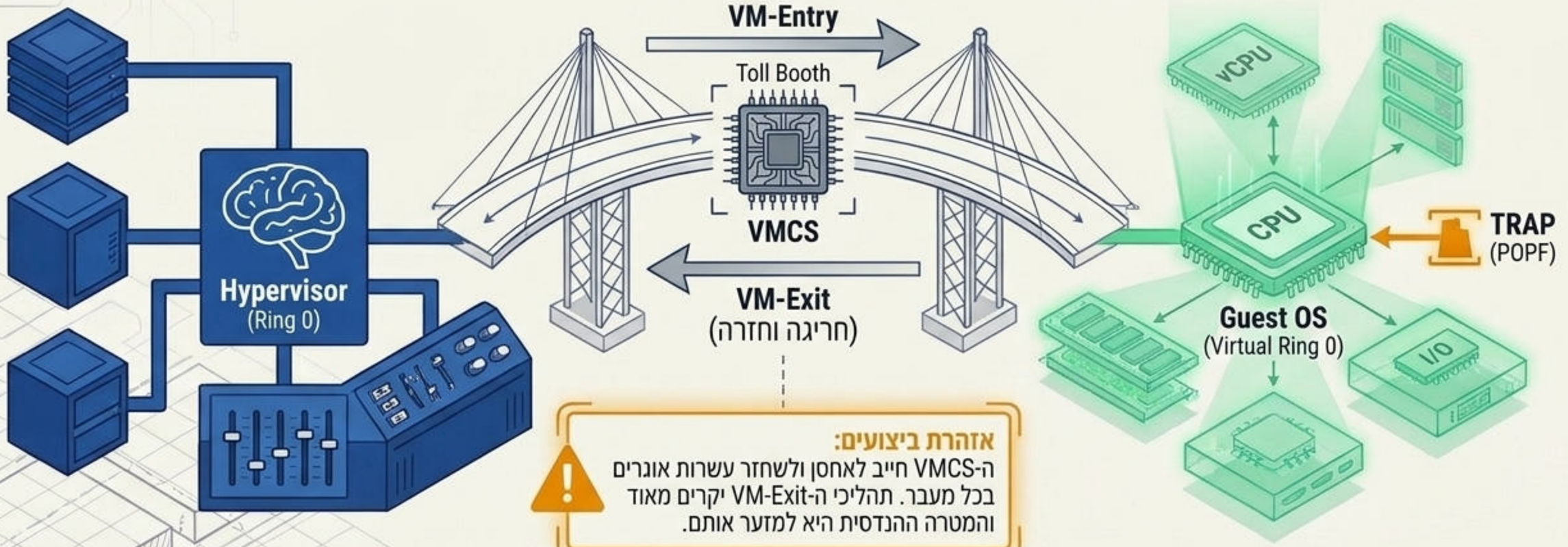
הפתרון החומרתי: פיצול המעבד (VT-x / AMD-V)

עולם האשליה (VMX Non-Root Operation)

כאן פועלת האורחת. מקבלת "Ring 0 אשלייתי" משלה.
פקודות כמו POPF מייצרות כעת TRAP חומרתי קשיח ללא שינוי קוד.

העולם האמיתי (VMX Root Operation)

כאן פועל ה-Hypervisor (Ring 0) עם הרשאות מוחלטות ושליטה מלאה.

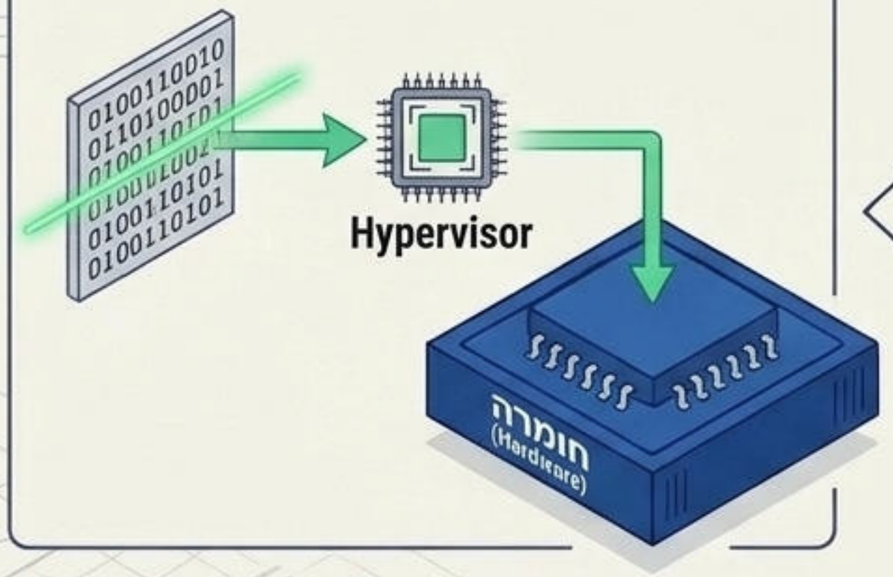


אזהרת ביצועים:
ה-VMCS חייב לאחסן ולשחזר עשרות אוגרים בכל מעבר. תהליכי ה-VM-Exit יקרים מאוד והמטרה ההנדסית היא למזער אותם.

https://operating-system-5l3.pages.dev/

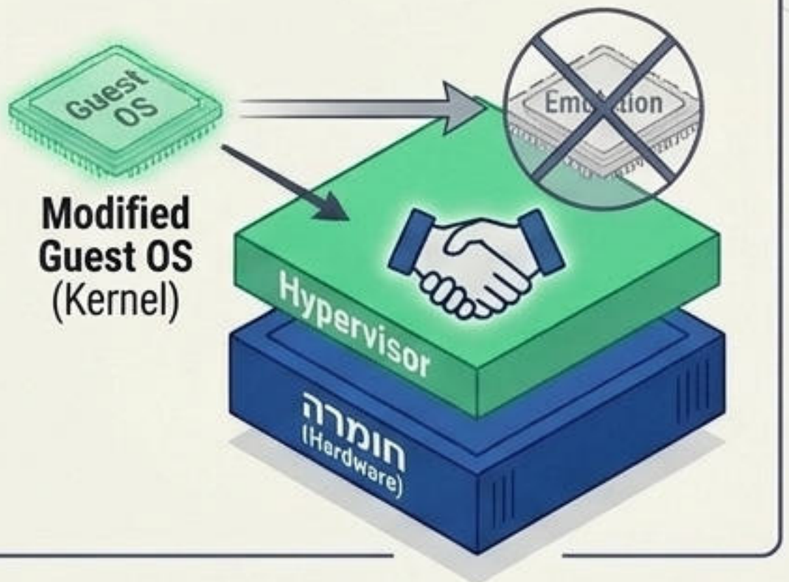
עוקפי חומרה: הפתרונות התוכנתיים של העידן הישן

A
1. תרגום בינארי דינמי (Dynamic Binary Translation)



- **ביצוע ישיר** (Direct): חישובים רגילים רצים ישירות על המעבד.
- **תרגום בזמן אמת**: ה-Hypervisor סורק את הקוד, מזהה הוראות רגישות ומחליף אותן.
- **מטמון קוד** (Code Cache): הבלוקים המתורגמים נשמרים בזיכרון לריצה עתידית מהירה מבלי לסרוק מחדש.

B
2. פרא-וירטואליזציה (Paravirtualization)

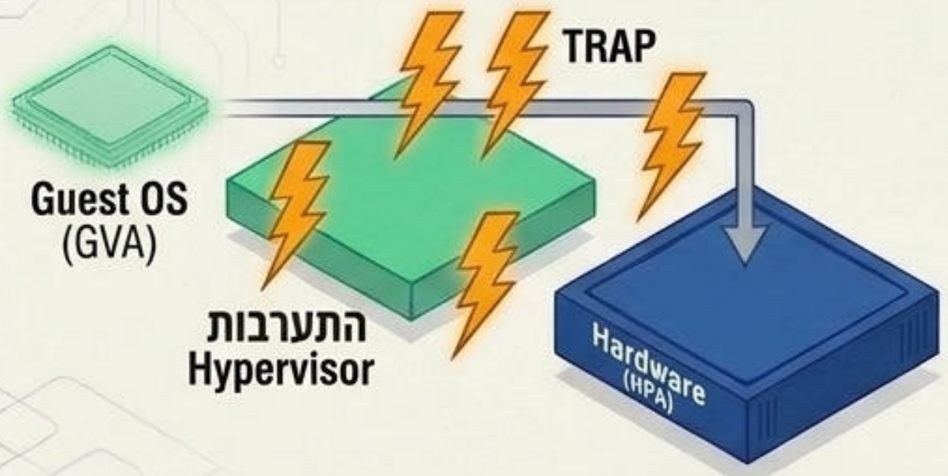


- **שיתוף פעולה:** ויתור על האשליה המושלמת. שינוי קוד המקור של ה-Guest OS כך שתדע שהיא וירטואלית.
- **Hypercalls**: החלפת הוראות רגישות בקריאות API ישירות ומסודרות ל-Hypervisor, מה שמונע TRAPs יקרים.
- אידיאלי לארכיטקטורת מיקרו-קרנל, מאפשר ל-Hypervisor להיות רזה ומהיר.

האתגר הכבד ביותר: וירטואליזציה של זיכרון

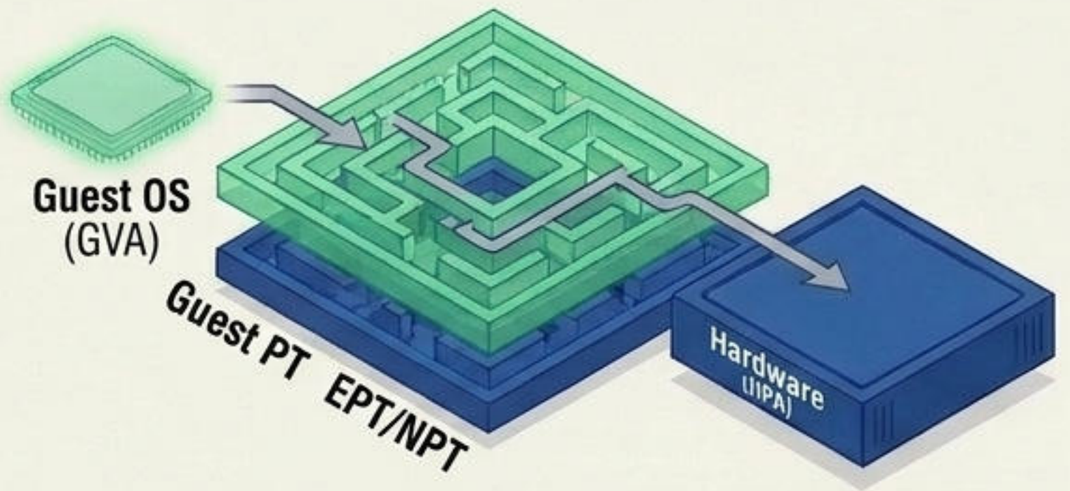
(כתובת וירטואלית של האורח) **GVA** ⟹ (כתובת פיזית של האורח) **GPA** ⟹ (כתובת פיזית אמיתית) **HPA**

גישת התוכנה - Shadow Page Tables



- ה-Hypervisor מחזיק "טבלת צל" הממפה ישירות מ-GVA ל-HPA.
- כל שינוי בטבלאות האורחת דורש פסיקת סנכרון (מאות אלפי TRAPs בשנייה המשתקים את המערכת).

הגישה החומרתית - EPT / NPT



- ה-MMU עובד עם 2 טבלאות נפרדות בו-זמנית ללא פסיקות.
- ה-GPA עובר אוטומטית דרך טבלת ה-EPT החומרתית למציאת ה-HPA.

המחיר הנסתר של ההליכה הדו-מימדית (2D Page Walk)

במקרה של TLB Miss, תרגום כתובת בודדת דורש הליכה במבוך המקונן העולה עד 24 גישות פיזיות לזיכרון!

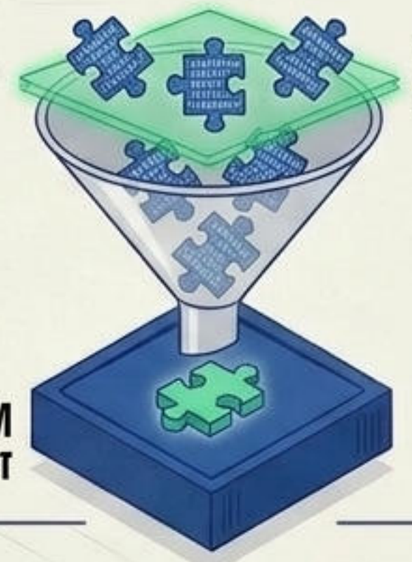
הפתרון: תיוג VPID המונע מחיקת ה-TLB במעבר בין המארח לאורח.

https://operating-system-5l3.pages.dev/

התחייבות יתר (Overcommitment): הנשק הסודי של הענן

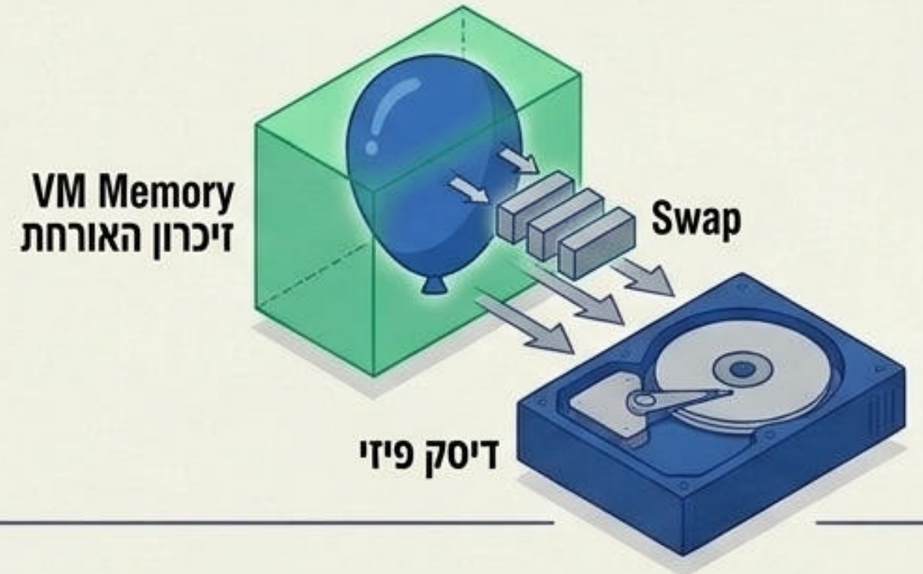
כיצד מוכרים יותר זיכרון ממה שקיים בשרת הפיזי?

A 1. מניעת כפילויות (KSM / Deduplication)



- ה-Hypervisor סורק את הזיכרון וממזג דפים זהים (לפי פונקציית Hash) למסגרת פיזית אחת (Read-Only).
- **Copy-On-Write**: הפרדה פיזית תתרחש מחדש רק במידה ואחת המכונות תנסה לשנות את הדף המשותף.
- **תורם לחיסכון פסיבי** של עשרות ג'יגה-בתים בשרת מארח.

B 2. התנפחות (Ballooning)



- כשיש מצוקת משאבים, ה-Hypervisor דורש מדרייבר ה'בלון' באורחת להתנפח.
- **הבלון נוטל זיכרון** מהאורחת, שנאלצת לפנות נתונים לדיסק, ומעביר את המסגרות הריקות בחזרה למארח.
- **סכנה** (OOM Killer): התנפחות אגרסיבית מדי עלולה להרוג תהליכים קריטיים במערכת ההפעלה האורחת.

https://operating-system-5l3.pages.dev/

ספקטרום הקלט/פלט (I/O Virtualization)



דימוי תוכנתי מלא של חומרה מיושנת.
ביצועים איומים עקב ריבוי Context Switches.
הכל עובר דרך ה-Hypervisor.



דרייבר מודע-וירטואליזציה.
דחיסת בקשות לתורי-טבעת משותפים (Batching) במטרה לרכז פסיקות.
ביצועים טובים.



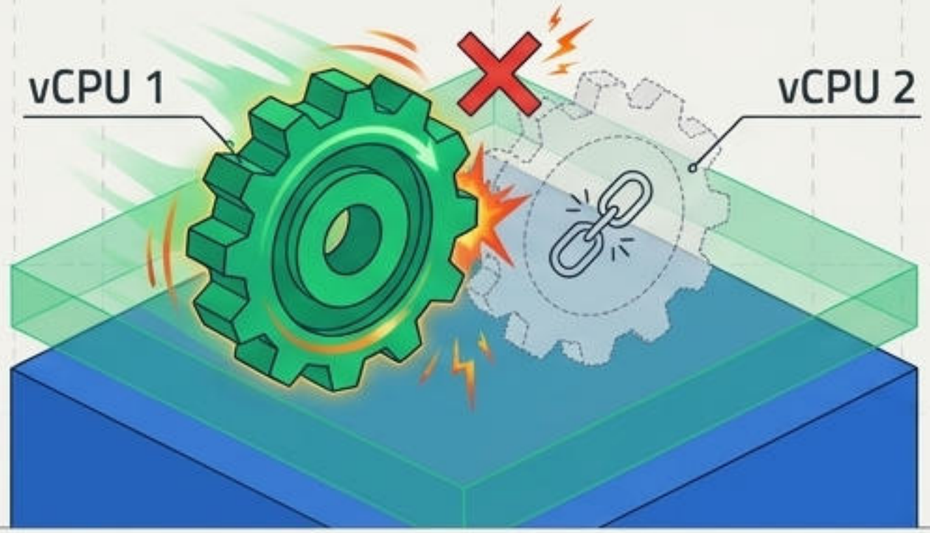
שיוך בלעדי של כרטיס פיזי ל-VM יחיד ועקיפת ה-VMM לחלוטין. חובה להשתמש בבקר IOMMU חומרתי לתרגום ולמניעת פרצות DMA.



הסטנדרט האולטימטיבי:
כרטיס רשת מודרני המתפצל פנימית לעשרות פונקציות וירטואליות בחומרה. הפעלת I/O עצמאית ומקבילה.

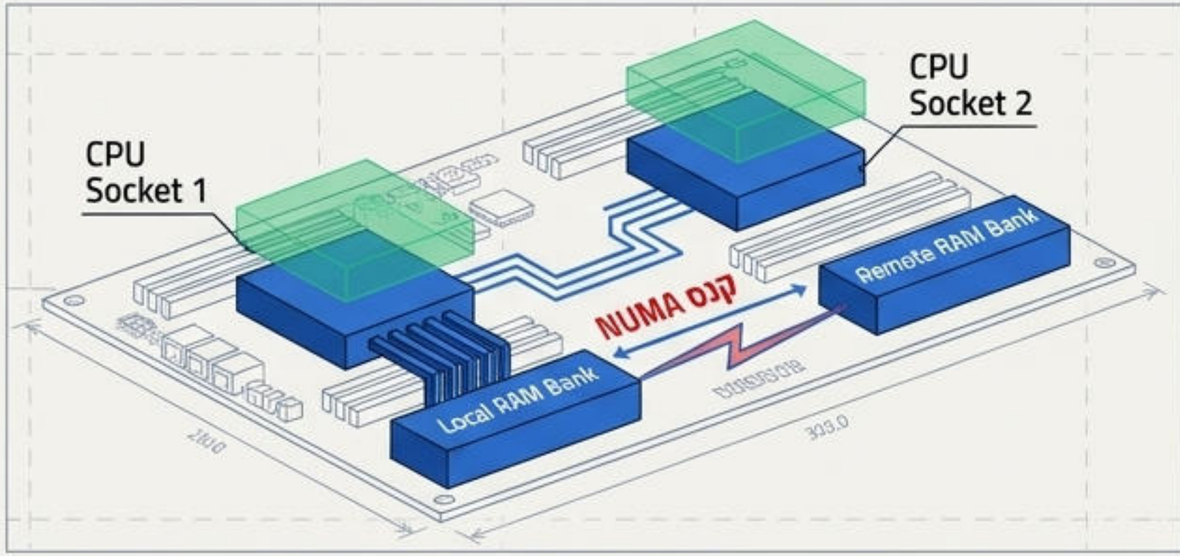


אתגרי תזמון בעולם מרובה מעבדים (SMP)



תזמון כנופיה ובעיית מנעולי הספין

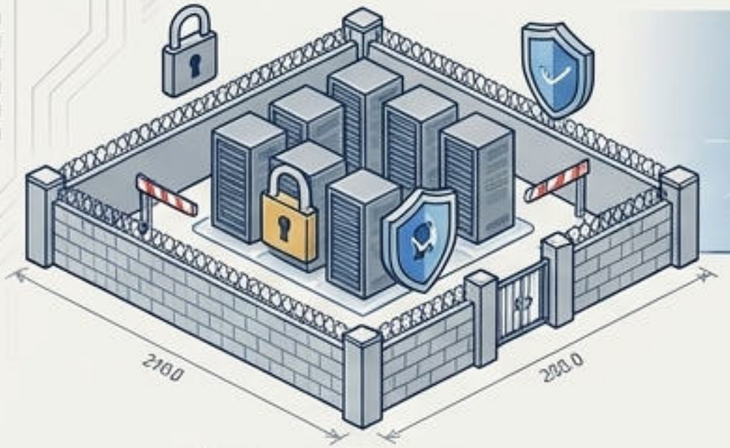
- **הבעיה (Spinlocks):** אם ה-Hypervisor משהה vCPU אחד, vCPU פעיל התלוי בו נתקע בלולאת ספין אינסופית ששורפת משאבי CPU לחינם.
- **הפתרון (Gang Scheduling):** התחייבות ארכיטקטונית של ה-VMM להריץ את כל ה-vCPU של אותה המכונה ביחד בדיוק באותו שבריר שנייה, או להשהות את כולם יחד.



טופולוגיית NUMA והצמדה

- **הבעיה:** זמן הגישה של מעבד לזיכרון אינו אחיד ('קנס NUMA' כאשר ניגשים לזיכרון של מעבד רחוק).
- **הפתרון** (NUMA Pinning)**:** הצמדה קשיחה של המעבדים הווירטואליים ומסגרות ה-RAM לאותו מעבד פיזי ולבנק המקומי שלו למניעת עיכובי חומרה.

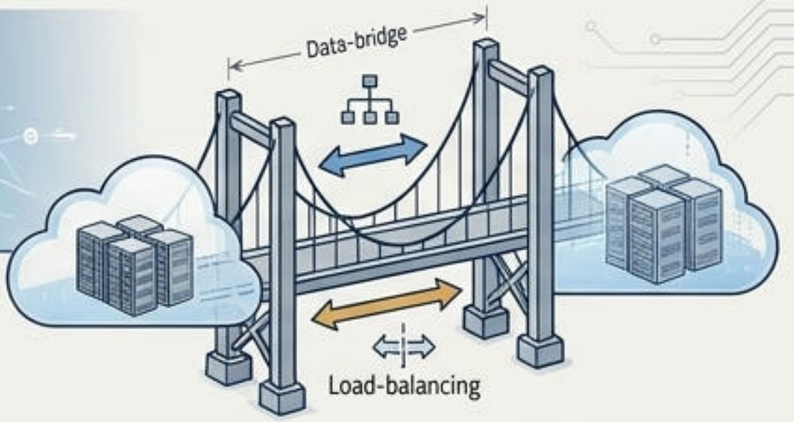
המעבר למחשוב ענן: מודלים ותכונות יסוד



ענן פרטי (Private)
אבטחה ופרטיות מוגברת, שליטה מלאה.



ענן ציבורי (Public)
סקייל אינסופי, תשלום לפי צריכה (Pay-as-you-go).



ענן היברידי (Hybrid)
גמישות מירבית, מאפשר זליגת עומסים.

5 תכונות היסוד של מחשוב ענן (תקן NIST)



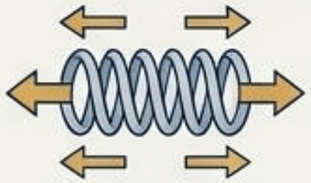
שירות עצמי (On-Demand)
הקצאה אוטומטית ע"י הלקוח.



גישה רחבה (Broad Access)
נגישות רשתית מלאה מכל מקום.



איגום משאבים (Resource Pooling)
מודל Multi-tenant משותף.

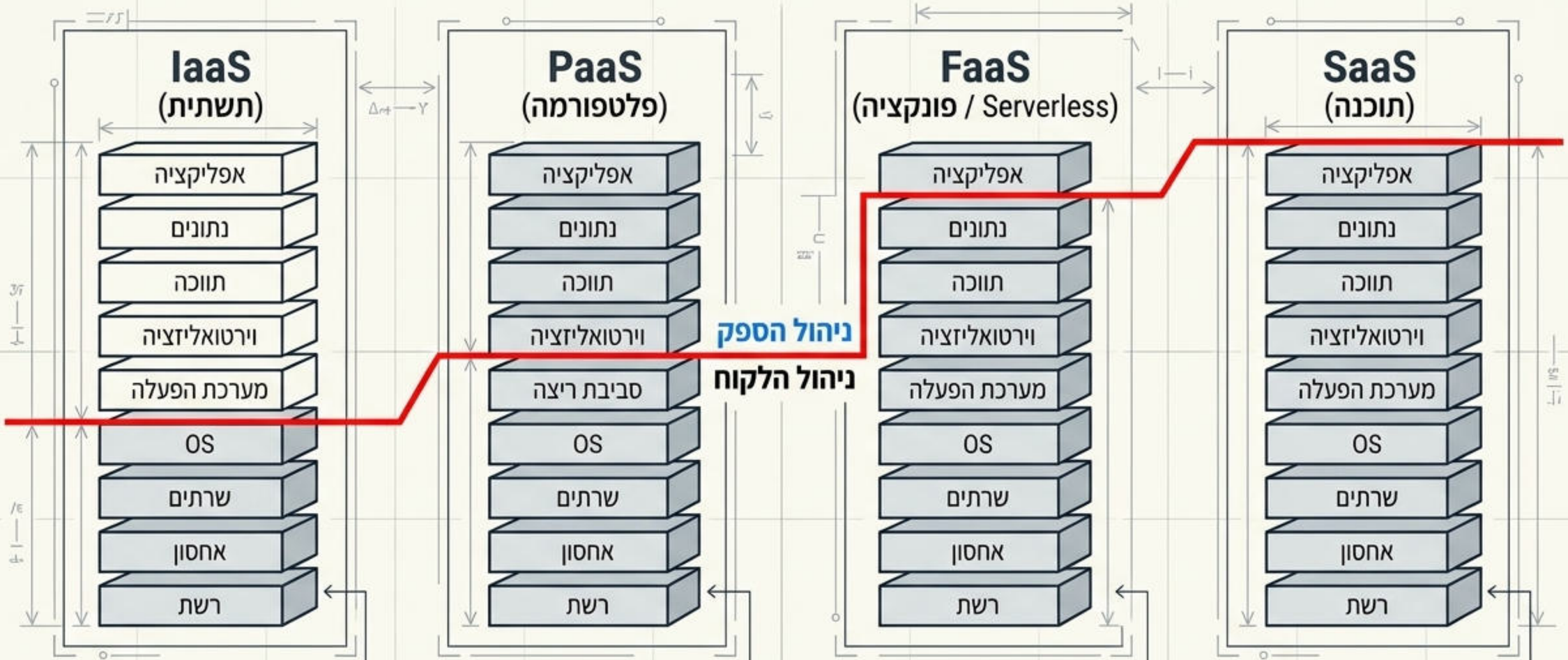


גמישות מהירה (Rapid Elasticity)
כיווץ והתרחבות מיידיים (Scale).



שירות מדיד (Measured Service)
חיוב וניטור מבוססי צריכה מדוייקת.

מודלי שירות בענן: קו האחריות (XaaS)



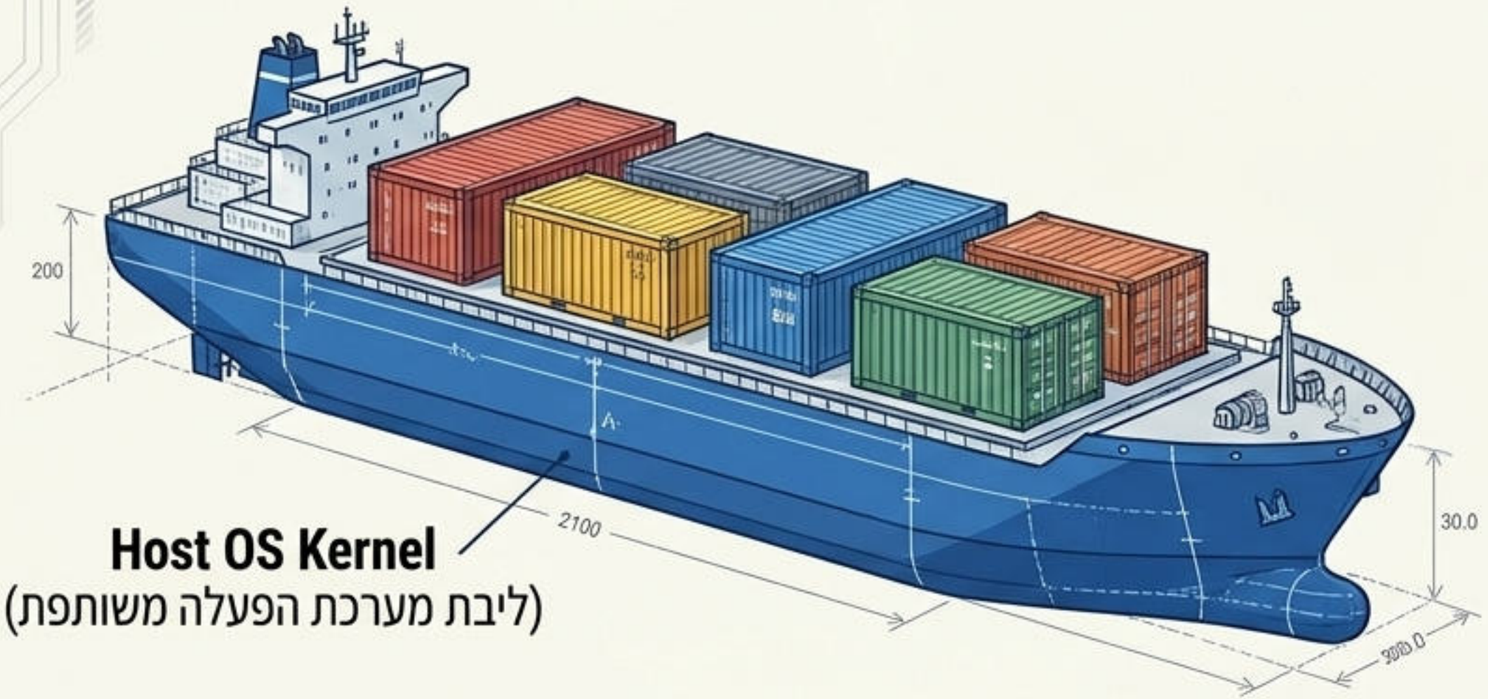
דוגמה: Amazon EC2
הלקוח מנהל: מערכת הפעלה, עדכונים, אבטחה, קוד.

דוגמה: Heroku
הלקוח מנהל: קוד אפליקציה בלבד. **הספק מנהל:** שרתים וסביבת ריצה.

דוגמה: AWS Lambda
מודל מונחה אירועים.
המפתח מעלה קטעי קוד, החיוב הוא לפי זמן ריצת הטריגר.

דוגמה: Gmail / Office 365
המוצר המוגמר.
הספק מנהל הכל, הלקוח רק משתמש.

קונטיינרים: המהפכה ברמת מערכת ההפעלה



Host OS Kernel
(ליבת מערכת הפעלה משותפת)

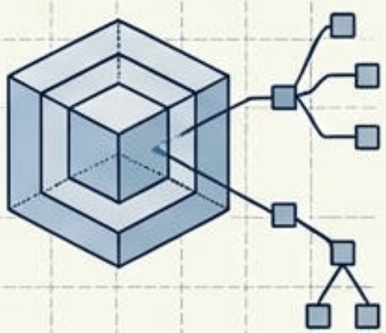
גישה הפוכה ממכונות
וירטואליות: במקום לחקות קיימת מערכת הפעלה אחת המפצלת את עצמה לסביבות לוגיות הרצות ישירות על הגרעין המשותף.

קבוצות בקרה (cgroups)



הבידוד הפיזי: האכיפה.
מנגנון העוקב, מגביל ומתעדף משאבי חומרה (מקס' ?Ω% % CPU ו-RAM) למכולה, למניעת הרעבת משאבים של קונטיינרים שכנים.

מרחבי שמות (Namespaces)



הבידוד הלוגי: יצירת האשליה.
כל קונטיינר מקבל מרחב משלו למספרי תהליכים (PIDs), עץ תיקיות, וכתובות רשת IP.
הוא עיוור לחלוטין לנעשה מחוץ לו.

סיכום ארכיטקטוני: מכונות וירטואליות מול קונטיינרים

קריטריונים (Criteria)	  **מכונות וירטואליות** (Virtual Machines)	  **קונטיינרים** (Containers)
משקל וזמן טעינה (Boot)	**כבד** (GBs). תקורת Boot ארוכה עד לטכיוו, לטעינת OS מלאה.	**קל משקל** (MBs). עולה בשבריר שנייה עולה בשבריר שנייה ללא תהליך Boot.
ניצול משאבים (Utilization)	**בזבוז יחסי** - כל מכונה טוענת עותק **Kernel** שלם משלה לזיכרון.	**יעילות עילאית**. אין כפילויות **Kernel**, **Kernel**,מאפשר הרצת אלפי עותקים.
תאימות מערכת (OS Compatibility)	**חופש מוחלט**. הרצת Windows ו-Linux ו-Linux במקביל על שרת יחיד.	**מוגבל** למשפחת הליבה המארחת (לרוב Linux בלבד).
בידוד ואבטחה (Isolation / Security)	**הפרדה חומרתית קשיחה**. קריסת קריסת VM אינה משפיעה על השכנים.	**בידוד ליבה משותף**. פריצה או קריסת קריצה או Kernel יפילו את השרת כולו.

סינרגיה מודרנית: תוכנה מודרנית רצה על קונטינרים (Docker/Kubernetes), אך אלו נפרסים לרוב על גל גבי תשתית **ברזל וירטואלית** (IaaS VMs) ברמת ספק הענן מסיבות אבטחה וניהול.

https://operating-system-5l3.pages.dev/